BudgetBuddy — Milestone 11: Measure Performance

Course: DSBA 6010 · Graduate LLM Applications
Group: 3 — Budget Buddy
Date: March 31, 2026

Table of Contents

1. Overview
2. System Under Evaluation
3. Evaluation Metrics
4. A/B Testing
5. Evaluation Tools
6. Scenario Coverage and Edge Cases
7. Results
8. Human Evaluation Protocol
9. Conclusions

---

1. Overview

BudgetBuddy is an LLM-powered personal finance assistant that features a multi-agent Pet Community Café. Four AI personas — Mochi (cat), Penny (penguin), Capy (capybara), and Esper (dragon) — hold grounded, turn-based conversations about the user's real budget data and live Reddit posts. This report documents how Milestone 11's performance-measurement requirements are satisfied across that system.

The evaluation framework lives in `backend/cafe_evaluation.py` and covers four automated metrics, a head-to-head A/B test, four scenario runs spanning normal and adversarial conditions, and a structured human-evaluation template.

---

2. System Under Evaluation

2.1 Architecture

```
POST /api/cafe/gossip
        │
        ▼
  run_cafe_continue_turn()          ← cafe_agents.py
        │
        ├─ load_cafe_memory()       ← cafe_tools.py  (JSON persistence)
        ├─ fetch_cafe_context()     ← cafe_tools.py  (budget + Reddit)
        ├─ _generate_turn()         ← OpenAI gpt-4o-mini → Anthropic → mock
        └─ save_cafe_memory()       ← cafe_tools.py
```

The primary agent entry point called by tests and the REST endpoint is `run_cafe_continue_turn`. Each click produces exactly one new message from the next speaker in the rotation: `user_pet → npc_pet_1 → npc_pet_2 → barista_planner`, with a no-repeat-in-two rule enforced.

2.2 Agent Personas

Key	Name	Role
`user_pet`	Mochi 🐱	Sweet, anxious cat; emotional user proxy
`npc_pet_1`	Penny 🐧	Warm, practical penguin; gives concrete tips
`npc_pet_2`	Capy 🦫	Protective capybara; firm budget guardian
`barista_planner`	Esper 🐉	Sassy dragon planner; issues QUESTs

2.3 Context Grounding

Each session starts with a call to `fetch_cafe_context`, which aggregates:

- **Budget data** — deterministic per `user_id` (over-spent / balanced / saved scenario)
- **Reddit posts** — live from the Reddit public JSON API with fallback fixtures

```
def fetch_cafe_context(user_id: str, subreddit: str = "coffee") -> Dict[str, Any]:
    budget = json.loads(fetch_budget_data(user_id))
    posts = fetch_reddit_posts(subreddit=subreddit, limit=3)
    return {
        "budget": budget,
        "reddit_posts": posts,
        "subreddit": subreddit,
        "fetched_at": datetime.now(UTC).isoformat(),
    }
```

The context summary is cached in the opening turn's `meta` field so every subsequent turn references the same grounded topic.

---

3. Evaluation Metrics

All four metrics are implemented as pure functions in `backend/cafe_evaluation.py`. Each takes a `List[Dict]` conversation history and returns a `float` in `[0.0, 1.0]`.

3.1 Accuracy

Definition: Structural and turn-management correctness — valid speaker keys, non-empty content, timestamp presence, and the no-repeat-in-two-turns rule.

```
def metric_accuracy(history: List[Dict[str, Any]]) -> float:
    """Structural and turn-management accuracy."""
    checks = 0
    passed = 0
    for i, entry in enumerate(history):
        checks += 1
        if entry.get("speaker") in ALLOWED_SPEAKERS:
            passed += 1
        checks += 1
        if isinstance(entry.get("content"), str) and bool(entry["content"].strip()):
            passed += 1
```

```
        checks += 1
        if "timestamp" in entry:
            passed += 1
        if i >= 2:
            checks += 1
            speaker = entry.get("speaker")
            prev  = history[i - 1].get("speaker")
            prev2 = history[i - 2].get("speaker")
            if speaker not in {prev, prev2}:
                passed += 1
    return round(passed / max(checks, 1), 4)
```

3.2 Relevance

Definition: Topical linkage between consecutive turns (token overlap with the prior message) combined with finance-domain grounding (presence of keywords such as *budget*, *save*, *expense*, *quest*).

```
FINANCE_KEYWORDS = {"budget","save","savings","spend","money",
                    "quest","hoard","expense","cash"}

def metric_relevance(history):
    pair_scores = []
    for i in range(1, len(history)):
        prev_tokens = set(_tokenize(str(history[i-1].get("content",""))))
        curr_tokens = set(_tokenize(str(history[i].get("content",""))))
        overlap_score = 1.0 if len(prev_tokens & curr_tokens) > 0 else 0.0
        domain_score  = 1.0 if any(k in str(history[i]["content"]).lower()
                                   for k in FINANCE_KEYWORDS) else 0.0
        pair_scores.append((overlap_score * 0.5) + (domain_score * 0.5))
    return round(mean(pair_scores), 4)
```

3.3 Coherence

Definition: Utterance quality — responses must be non-empty, at most two sentences, and must not contain nested speaker-prefix chains (e.g., `"Capy: for Mochi: ..."`).

```
def metric_coherence(history):
    scores = []
    for entry in history:
        content = str(entry.get("content","")).strip()
        s = 0.0
        if content:                                    s += 0.4
        if _sentence_count(content) <= 2:              s += 0.3
        if not _has_nested_speaker_quotes(content):    s += 0.3
        scores.append(s)
    return round(mean(scores), 4)
```

Nested-quote detection uses a regex scan:

```python
def _has_nested_speaker_quotes(text: str) -> bool:
    matches = re.findall(r"(?:Mochi|Penny|Capy|Esper):", text)
    return len(matches) > 1
```

3.4 Faithfulness

Definition: Consistency between message content and its metadata annotations — the `quest` flag in `meta` must agree with whether the word "quest" appears in the content, and at least one tool call (`fetch_budget_data`) must have been recorded in the session.

```python
def metric_faithfulness(history):
    checks = 0; passed = 0; saw_tool = False
    for entry in history:
        meta = entry.get("meta", {}) or {}
        checks += 1
        quest_flag = bool(meta.get("quest"))
        if ("quest" in str(entry.get("content","")).lower()) == quest_flag \
                or not quest_flag:
            passed += 1
        if meta.get("tool") == "fetch_budget_data":
            saw_tool = True
    checks += 1
    if saw_tool:
        passed += 1
    return round(passed / max(checks, 1), 4)
```

3.5 Composite Score

The overall score is the mean of all four metric scores across all scenarios and the A/B system variant.

---

4. A/B Testing

4.1 Design

Two variants are compared across 6 turns each:

Variant	Description
System	Full pipeline: `run_cafe_continue_turn` → OpenAI gpt-4o-mini → Anthropic Claude Haiku fallback
Baseline	Mock-only: `_mock_single_turn` with deterministic, topic-templated strings

```python
def run_ab_test(user_id, turns, memory_path):
    system_history   = run_system_variant(user_id, turns, memory_path)
    baseline_history = run_baseline_variant(turns)

    system_metrics   = evaluate_history(system_history)
    baseline_metrics = evaluate_history(baseline_history)

    delta = {k: round(system_metrics[k] - baseline_metrics[k], 4)
```

```
            for k in system_metrics}

    winner = "system" if mean(delta.values()) >= 0 else "baseline"
    return {
        "system_metrics":   system_metrics,
        "baseline_metrics": baseline_metrics,
        "delta":            delta,
        "winner":           winner,
        "turns":            turns,
    }
```

4.2 Baseline Construction

The baseline intentionally uses zero live context and no LLM calls:

```
def run_baseline_variant(turns: int) -> List[Dict[str, Any]]:
    transcript = [{
        "id": "baseline-0",
        "speaker": "barista_planner",
        "content": "Cafe open. Ledger synced — one focused topic at a time.",
        "meta": {"tool": "fetch_budget_data", "quest": False},
    }]
    for _ in range(max(0, turns - 1)):
        transcript.append(_baseline_turn(transcript))
    return transcript
```

Each `_baseline_turn` calls `_mock_single_turn(speaker, prev_content, turn_index)`, which inserts the context summary's first sentence into template strings, ensuring the baseline is non-trivial while being fully deterministic.

4.3 A/B Results

Metric	System	Baseline	Δ
Accuracy	1.0000	1.0000	0.0000
Relevance	0.9000	0.9000	0.0000
Coherence	1.0000	1.0000	0.0000
Faithfulness	1.0000	1.0000	0.0000
Winner	**System**	—	—

The system variant wins the tie-break by design (delta ≥ 0 → system). Equal scores on structured metrics confirm the mock baseline's structural correctness while the qualitative difference (topic grounding from live Reddit + real LLM generation) is captured by the human evaluation protocol in Section 8.

---

5. Evaluation Tools

5.1 Custom Test Suite (Primary)

The custom suite in `backend/cafe_evaluation.py` and `tests/test_cafe_evaluation.py` was chosen as the primary framework because:

- TruLens requires an OpenAI-compatible feedback LLM at evaluation time, adding external latency and cost.
- RAGAS is optimised for RAG pipelines with explicit question/context/answer triples, which do not map cleanly to a multi-turn chat scenario.
- The custom suite is stateless, fully reproducible, and runs in under one second without any API calls.

The dynamic detection code for external tools runs at suite start:

```
def detect_external_eval_tools() -> Dict[str, bool]:
    tools = {"trulens": False, "ragas": False}
    try:
        __import__("trulens_eval")
        tools["trulens"] = True
    except Exception:
        pass
    try:
        __import__("ragas")
        tools["ragas"] = True
    except Exception:
        pass
    return tools
```

At the time of the last evaluation run both were `false`, confirming the in-house approach is the sole active framework.

5.2 pytest Integration

Both test files run under `pytest` with no additional dependencies:

```
# Run all cafe evaluation tests
python -m pytest tests/test_cafe_agents.py tests/test_cafe_evaluation.py -v
```

`tests/test_cafe_agents.py` validates multi-step conversation, tool orchestration, and quest output:

```
def test_cafe_agents_multi_step_tool_and_quest(tmp_path):
    history = run_cafe_group_chat(
        user_id="integration_test_user",
        memory_path=str(tmp_path / "cafe_memory.json"),
        max_round=6,
    )
    assert len(history) >= 4
    speakers = {e.get("speaker") for e in history}
    assert len(speakers & {"user_pet","npc_pet_1","npc_pet_2","barista_planner"}) >= 3
    assert any(e.get("meta",{}).get("tool") == "fetch_budget_data" for e in history)
    assert "quest" in str(history[-1].get("content","")).lower()
```

`tests/test_cafe_evaluation.py` validates that the evaluation suite produces all three output artifacts with the expected schema:

```python
def test_cafe_evaluation_suite_generates_reports(tmp_path):
    payload = run_evaluation_suite(str(tmp_path))
    assert 0.0 <= float(payload["overall_score"]) <= 1.0
    assert {"accuracy","relevance","coherence","faithfulness"}.issubset(
        payload["ab_test"]["system_metrics"])
    assert {"happy_path","corrupted_memory_recovery",
            "long_nested_context","no_api_key_mode"}.issubset(
        {s["name"] for s in payload["scenarios"]})
    assert (tmp_path / "docs" / "cafe_evaluation_results.json").exists()
    assert (tmp_path / "docs" / "CAFE_EVALUATION_REPORT.md").exists()
    assert (tmp_path / "docs" / "cafe_human_eval_template.csv").exists()
```

---

6. Scenario Coverage and Edge Cases

Four named scenarios are run as part of every evaluation suite execution. Each gets its own isolated memory file so scenarios cannot interfere with each other.

```python
scenarios = [
    EvaluationScenario("happy_path",
        "Standard multi-turn cafe flow.", turns=6),
    EvaluationScenario("corrupted_memory_recovery",
        "Start from malformed-memory-like minimal state.", turns=5,
        seed_history=[{"id":"seed","speaker":"barista_planner",
                       "content":"...","timestamp":"old","meta":{}}]),
    EvaluationScenario("long_nested_context",
        "Stress with nested speaker-like prefixes in prior content.", turns=5,
        seed_history=[{"id":"seed","speaker":"npc_pet_1",
                       "content":"Capy: for Mochi: think about your budget",
                       "timestamp":"old","meta":{}}]),
    EvaluationScenario("no_api_key_mode",
        "Validation when live LLM is not configured.", turns=4),
]
```

6.1 Happy Path

Normal six-turn conversation starting from an empty history. Validates the full pipeline: context fetch → Esper opening → speaker rotation → LLM generation → memory persistence.

Pass criteria: All four metrics ≥ 0.85; no failure modes.

6.2 Corrupted Memory Recovery

Seeds the memory file with a minimal, structurally incomplete prior turn that mimics a truncated or partially-written JSON file. Tests that `load_cafe_memory` returns the seed gracefully and that `_generate_turn` continues without crashing.

Failure mode probed: `turn_rule_violation` (accuracy < 0.85).

6.3 Long / Nested Context

Seeds the conversation with content containing literal nested speaker prefixes (`"Capy: for Mochi: ..."`). This is the primary adversarial case for the `_clean_content` content-cleaning function:

```
def _clean_content(content: str) -> str:
    """Strip any speaker-prefix nesting, return last meaningful sentence."""
    parts = re.split(r"\b(Mochi|Penny|Capy|Esper):\s*", content)
    speaker_names = {"Mochi","Penny","Capy","Esper"}
    for segment in reversed(parts):
        segment = segment.strip()
        if segment and segment not in speaker_names:
            return segment
    return content.strip()
```

Failure mode probed: `nested_quotes_detected` (coherence impact).

6.4 No-API-Key Mode

Runs without an `OPENAI_API_KEY` environment variable set, exercising the mock fallback path in `_generate_turn` :

```
def _generate_turn(speaker, previous_content, context_summary, turn_index):
    ...
    text = _call_openai(...) or _call_anthropic(...)
    if not text:
        return _mock_single_turn(speaker, context_summary, turn_index)
    ...
```

Failure mode probed: Graceful degradation — verifies that the mock fallback produces structurally valid messages even when all live LLM providers are unavailable.

6.5 Failure Mode Taxonomy

Mode	Trigger	Action
`coherence_drop`	coherence < 0.7	Flag scenario as `needs_review`
`turn_rule_violation`	accuracy < 0.85	Flag scenario as `needs_review`
`nested_quotes_detected`	regex match > 1 speaker prefix in any message	Flag scenario as `needs_review`

---

7. Results

Full results are persisted to `docs/cafe_evaluation_results.json` on every suite run.

7.1 Scenario Metrics

Scenario	Accuracy	Relevance	Coherence	Faithfulness	Status
happy_path	1.0000	0.9000	1.0000	1.0000	✅ passed
corrupted_memory_recovery	1.0000	0.8750	1.0000	0.8333	✅ passed

long_nested_context	1.0000	0.8750	1.0000	0.8333	✅ passed
no_api_key_mode	1.0000	0.8333	1.0000	1.0000	✅ passed

7.2 Overall Score

$$\text{Overall} = \frac{1}{N}\sum_{i}\frac{\text{acc}_i + \text{rel}_i + \text{coh}_i + \text{fth}_i}{4} = \mathbf{0.9469}$$

7.3 Observations

- **Accuracy = 1.00 across all scenarios.** The speaker rotation rule (no repeat in last two turns) and schema validation have zero violations.
- **Coherence = 1.00 across all scenarios.** The `_clean_content` pipeline strips nested prefixes before they reach the LLM prompt, and the 1–2 sentence hard constraint in the system prompt prevents long responses.
- **Relevance is the most variable metric (0.83–0.90).** Token-overlap with a prior turn is inherently lower when the `_mock_single_turn` path is taken (no-API-key scenario), because the mock templates share fewer lexical tokens with the LLM-generated preceding turn.
- **Faithfulness drops to 0.83 in recovery/nested scenarios.** These scenarios use a non-standard seed history where the first turn does not have `meta.tool = "fetch_budget_data"`, so the faithfulness check's tool-presence sub-criterion is penalised.

---

8. Human Evaluation Protocol

8.1 Template

The evaluation suite generates `docs/cafe_human_eval_template.csv` automatically:

```
def generate_human_eval_template(path, sample_history):
    headers = ["turn_index","speaker","content",
               "relevance_1_to_5","coherence_1_to_5",
               "faithfulness_1_to_5","overall_1_to_5","notes"]
    with open(path,"w",newline="",encoding="utf-8") as handle:
        writer = csv.writer(handle)
        writer.writerow(headers)
        for i, entry in enumerate(sample_history):
            writer.writerow([i, entry.get("speaker",""),
                             entry.get("content",""), "","","","","",""])
```

8.2 Scoring Rubric

Each annotator scores each turn on four 1–5 Likert dimensions:

Dimension	1	3	5
Relevance	Completely off-topic	Loosely related to prior turn	Directly builds on prior context
Coherence	Rambling, nested quotes	Mostly clear, minor issues	Concise, plain, no artifacts
Faithfulness	Quest flag disagrees with text	Partially aligned	Fully consistent with metadata

Overall	Poor; disrupts experience	Acceptable	Excellent; feels natural

8.3 Recommended Study Design

- **Annotators:** 3 independent raters per conversation sample
- **Sample size:** 10 full six-turn conversations (60 turns total)
- **Inter-rater reliability:** Cohen's κ target ≥ 0.6
- **Comparison:** Human overall scores vs. automated composite score to calibrate threshold values

---

9. Conclusions

The BudgetBuddy Pet Café multi-agent system achieves an automated composite evaluation score of **0.9469** across four metrics (accuracy, relevance, coherence, faithfulness) and four diverse test scenarios. Key findings:

1. **Structural correctness is robust.** Speaker rotation rules and schema validation have zero failures across all scenarios and the A/B test.
2. **Coherence is fully maintained.** The combination of prompt-level hard constraints and the `_clean_content` pre-processing pipeline eliminates nested speaker-prefix artifacts in all tested conditions.
3. **Graceful degradation works.** The no-API-key scenario confirms that the mock fallback produces structurally valid, topically grounded output when live LLM providers are unavailable.
4. **Faithfulness is sensitive to seed state.** Scenarios that inject atypical seed histories lower faithfulness because the tool-call presence criterion cannot be retroactively satisfied; this is by design.
5. **The A/B framework is in place** and ready to report meaningful deltas as soon as live LLM output diverges structurally from the mock baseline on new model versions or prompt changes.

The framework is extensible: if TruLens or RAGAS packages are installed into the virtual environment, `detect_external_eval_tools()` will automatically report them and they can be called alongside the custom metrics with no changes to the test runner.

Report generated from `docs/cafe_evaluation_results.json` — last run: March 31, 2026.